파일 무결성 검증 실습

Windows 10/11 + VS Code + cl.exe 환경에서 **SHA-256 해시값을 계산하고 두 파일의 해시값을 비교**하도록 구성하면 적합합니다.

실습 목표

원본 파일과 복사된 증거 파일의 SHA-256 값을 계산하여 다음과 같이 확인합니다.

원본 파일
↓ SHA-256 해시값 A
복사 파일
↓ SHA-256 해시값 B

A == B → 파일 무결성 유지
A != B → 파일 변경 가능성

C 코드 — file_integrity.c (1/2)

```c
#define WIN32_LEAN_AND_MEAN
#include <windows.h>
#include <bcrypt.h>
#include <stdio.h>
#pragma comment(lib, "bcrypt.lib")

#define BUFFER_SIZE 4096

/*
 * 파일의 SHA-256 해시값을 계산하는 함수
 *
 * fileName  : 검사할 파일 이름
 * hashValue : 계산된 SHA-256 값을 저장할 배열
 *
 * 성공하면 1
 * 실패하면 0 반환
 */
int calculateSHA256(
    const char *fileName,
    unsigned char hashValue[32])
{
    HANDLE hFile = INVALID_HANDLE_VALUE;
    BCRYPT_ALG_HANDLE hAlgorithm = NULL;
    BCRYPT_HASH_HANDLE hHash = NULL;
    NTSTATUS status;
    DWORD hashObjectSize = 0;
    DWORD dataSize = 0;
    unsigned char *hashObject = NULL;
    BYTE buffer[BUFFER_SIZE];
    DWORD readBytes;

    /*
     * ------------------------------------------------
     * 1. 검사할 파일 열기
     * ------------------------------------------------
     *
     * GENERIC_READ를 사용하므로
     * 파일을 읽기 전용으로 접근한다.
     */
    hFile = CreateFileA(
        fileName,
        GENERIC_READ,
        FILE_SHARE_READ,
        NULL,
        OPEN_EXISTING,
        FILE_ATTRIBUTE_NORMAL,
        NULL
    );
    if (hFile == INVALID_HANDLE_VALUE) {
        printf( "파일 열기 실패 : %s\n", fileName );
        printf( "Error Code : %lu\n", GetLastError() );
        return 0;
    }
```

```c
/*
 * ------------------------------------------------
 * 2. SHA-256 알고리즘 공급자 열기
 * ------------------------------------------------
 */
status = BCryptOpenAlgorithmProvider(
&hAlgorithm,
BCRYPT_SHA256_ALGORITHM,
NULL,
0
);
if (status != 0) {
printf( "BCryptOpenAlgorithmProvider 실패\n" );
CloseHandle(hFile);
return 0;
}

/*
 * ------------------------------------------------
 * 3. SHA-256 해시 객체 크기 확인
 * ------------------------------------------------
 */
status = BCryptGetProperty(
hAlgorithm,
BCRYPT_OBJECT_LENGTH,
(PUCHAR)&hashObjectSize,
sizeof(DWORD),
&dataSize,
0
);
if (status != 0) {
printf( "BCryptGetProperty 실패\n" );
BCryptCloseAlgorithmProvider( hAlgorithm, 0 );
CloseHandle(hFile);
return 0;
}

/*
 * ------------------------------------------------
 * 4. 해시 객체를 저장할 메모리 할당
 * ------------------------------------------------
 */
hashObject = (unsigned char *)HeapAlloc(
GetProcessHeap(),
0,
hashObjectSize
);
if (hashObject == NULL) {
printf( "메모리 할당 실패\n" );
BCryptCloseAlgorithmProvider( hAlgorithm, 0 );
CloseHandle(hFile);
return 0;
}
```

C 코드 — file_integrity.c (2/2)

```c
/*
 * ---------------------------------------------
 * 5. SHA-256 Hash 객체 생성
 * ---------------------------------------------
 */
status = BCryptCreateHash(
    hAlgorithm,
    &hHash,
    hashObject,
    hashObjectSize,
    NULL,
    0,
    0
);
if (status != 0) {
    printf( "BCryptCreateHash 실패\n" );
    HeapFree( GetProcessHeap(), 0, hashObject );
    BCryptCloseAlgorithmProvider( hAlgorithm, 0 );
    CloseHandle(hFile);
    return 0;
}

/*
 * ---------------------------------------------
 * 6. 파일 내용을 일정 크기로 반복해서 읽는다.
 * ---------------------------------------------
 *
 * 전체 파일을 한 번에 메모리에 올리지 않고
 * 4096 Byte 단위로 읽는다.
 */
while (1) {
    if (!ReadFile(
        hFile,
        buffer,
        BUFFER_SIZE,
        &readBytes,
        NULL)) {
        printf( "파일 읽기 실패\n" );
        BCryptDestroyHash( hHash );
        HeapFree( GetProcessHeap(), 0, hashObject );
        BCryptCloseAlgorithmProvider( hAlgorithm, 0 );
        CloseHandle(hFile);
        return 0;
    }
    /* 파일 끝에 도달 */
    if (readBytes == 0) { break; }

    /* 읽은 데이터를 SHA-256 계산에 추가 */
    status = BCryptHashData(
        hHash, buffer, readBytes, 0 );
    if (status != 0) {
        printf( "BCryptHashData 실패\n" );
        BCryptDestroyHash( hHash );
        HeapFree( GetProcessHeap(), 0, hashObject );
        BCryptCloseAlgorithmProvider( hAlgorithm, 0 );
        CloseHandle(hFile);
        return 0;
    }
}
```

```c
/*
 * ---------------------------------------------
 * 7. 최종 SHA-256 해시값 생성
 * ---------------------------------------------
 *
 * SHA-256 결과는 32 Byte이다.
 */
status = BCryptFinishHash(
hHash, hashValue, 32, 0 );
if (status != 0) {
printf( "BCryptFinishHash 실패\n" );
BCryptDestroyHash( hHash );
HeapFree( GetProcessHeap(), 0, hashObject );
BCryptCloseAlgorithmProvider( hAlgorithm, 0 );
CloseHandle(hFile);
return 0;
}

/* 사용한 자원 반환 */
BCryptDestroyHash( hHash );
HeapFree( GetProcessHeap(), 0, hashObject );
BCryptCloseAlgorithmProvider( hAlgorithm, 0 );
CloseHandle( hFile );
return 1;
}

/* SHA-256 해시값 출력 */
void printHash( const unsigned char hash[32]) {
int i;
for (i = 0; i < 32; i++) {
printf( "%02x", hash[i] );
}
printf("\n");
}

/* 두 SHA-256 해시값 비교 */
int compareHash(
const unsigned char hash1[32],
const unsigned char hash2[32]) {
int i;
for (i = 0; i < 32; i++) {
if (hash1[i] != hash2[i]) { return 0; }
}
return 1;
}

int main(void) {
char originalFile[MAX_PATH];
char evidenceFile[MAX_PATH];
unsigned char originalHash[32];
unsigned char evidenceHash[32];

SetConsoleOutputCP( CP_UTF8 );
printf( "========================================\n" );
printf( " Digital Forensic File Integrity Check\n" );
printf( " SHA-256 File Hash Verification\n" );
printf( "========================================\n\n" );

printf( "원본 파일 경로 입력 : " );
scanf_s( "%259s", originalFile,
(unsigned)sizeof(originalFile) );
printf( "비교 파일 경로 입력 : " );
scanf_s( "%259s", evidenceFile,
(unsigned)sizeof(evidenceFile) );
printf("\n");

if (!calculateSHA256( originalFile, originalHash))
return 1;
```

cl.exe 컴파일

Windows의 BCrypt API를 사용하므로 bcrypt.lib를 링크합니다.

```
cl /W4 /utf-8 file_integrity.c bcrypt.lib
```

코드에 이미 다음 구문이 있기 때문에 일반적으로 다음처럼도 가능합니다.

```
#pragma comment(lib, "bcrypt.lib")
```

따라서:

```
cl /W4 /utf-8 file_integrity.c
```

실습 방법

먼저 테스트 파일을 하나 준비합니다.

```
C:\Forensic\original.txt
```

내용:

```
Digital Forensic Evidence
Windows 11
2026
```

같은 파일을 복사하여:

```
C:\Forensic\evidence.txt
```

두 파일을 프로그램에 입력합니다.

```
========================================
 Digital Forensic File Integrity Check
 SHA-256 File Hash Verification
========================================

원본 파일 경로 입력 : C:\Forensic\original.txt
비교 파일 경로 입력 : C:\Forensic\evidence.txt

----------------------------------------
[원본 파일]
파일   : C:\Forensic\original.txt
SHA-256 : 8a46f82c...

[비교 파일]
파일   : C:\Forensic\evidence.txt
SHA-256 : 8a46f82c...
----------------------------------------
[검증 결과]
SHA-256 값이 같습니다.
파일의 무결성이 유지되었습니다.
----------------------------------------
```

그다음 evidence.txt에 글자 하나만 추가한 뒤 다시 실행하면 SHA-256 값이 달라집니다.

```
원본 파일 original.txt
 |
 ├──── SHA-256
 |     ▼
 8A46F82C...

복사된 증거 파일 evidence.txt
 |
 ├──── SHA-256
 |     ▼
 8A46F82C...
      ↓ 비교
 [일치] 증거 파일 무결성 유지
```

반대로 파일이 수정되면 다음과 같습니다.

```
Original SHA-256
 8A46F82C...
      |
      | X
      |
Evidence SHA-256
 5B921AD7...
→ 파일 변경 가능성 확인
```

이 실습에서 학생들이 이해해야 할 핵심은 **파일 이름이나 크기가 같다고 동일한 증거라고 판단하는 것이 아니라, 파일의 실제 데이터를 기반으로 생성된 SHA-256 해시값을 비교하여 무결성을 검증한다는 것**입니다.